

Experiment – 1.2.2

Student Name: Yatharth Bhaskar

UID: 24IBD70014

Branch: AIT-CSE (Data Science)

Section/Group: 24BDS-4 NTPP (A)

Semester: 5th

Date of Performance: 15 July 2026

Subject: Full Stack-II

Subject Code: 24CSP-337

1. Aim:

To optimize state access and improve application performance using memoized selectors and efficient rendering strategies.

2. Software Required:

- Node.js & npm
- React.js
- Redux Toolkit
- React-Redux
- Code Editor (VS Code)

3. Objective:

- To understand the concept of derived state
- To implement memoized selectors using Reselect
- To reduce unnecessary component re-renders
- To improve performance in large-scale applications

4. Theory:

In large-scale applications, inefficient state access and frequent re-renders can significantly degrade performance. Redux selectors are used to extract specific pieces of state; however, without optimization, they may recompute values unnecessarily.

Memoization is a technique used to cache computed results and return them only when input data changes. Libraries like Reselect provide memoized selectors that prevent redundant computations and reduce rendering overhead.

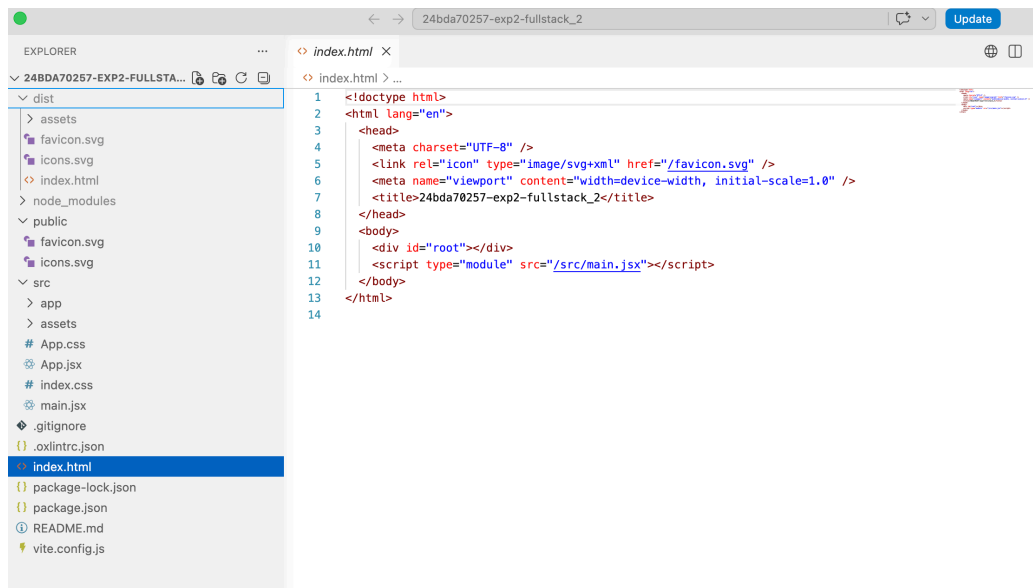
Derived state refers to data computed from the existing state rather than stored directly. By using selectors, applications can compute complex data efficiently without duplicating state.

Optimizing selectors and rendering behavior ensures that components update only when necessary, improving responsiveness and scalability.

5. Procedure/Algorithm:

- Define basic selectors for accessing state
- Implement memoized selectors using createSelector
- Compute derived data (e.g., filtered posts, grouped data)
- Integrate selectors into components
- Use React.memo and useMemo where necessary
- Analyze and minimize unnecessary re-renders

6. Code:



```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <title>24bda70257-exp2-fullstack_2</title>
8   </head>
9   <body>
10    <div id="root"></div>
11    <script type="module" src="/src/main.jsx"></script>
12  </body>
13 </html>
14
```

7. Output:

REDUX TOOLKIT + MEMOIZED SELECTORS

Centralized state for posts and platforms

This demo uses a normalized store, async loading, and memoized selectors to keep the UI fast and predictable.

Add sample post

Add platform

Reload posts

Post feed

Status: succeeded

Redux Toolkit overview · Web Platform

Remove

Normalized data pattern · Mobile Platform

Remove

Memoized selectors · Desktop Platform

Remove

Platform summary

Web Platform: 1

Mobile Platform: 1

Desktop Platform: 1

8. Conclusion:

- Efficient state access using selectors
- Reduced redundant computations
- Improved rendering performance
- Scalable frontend architecture

9. Learning Outcomes:

- Understand the role of selectors in accessing Redux state efficiently.
- Learn the concept of memoization to optimize state computations.
- Implement memoized selectors using Reselect.
- Compute derived state without duplicating data in the store.
- Improve React application performance by reducing unnecessary re-renders.